Sichtbarkeit und Lebensdauer

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Idee von Unterprogrammen
- Methoden in Java
- Parameter
- Rückgabe
- Überladen

Was wird in diesem Foliensatz besprochen?

- Sichtbarkeit von Variablen
- Lebensdauer von Variablen
- Lokale Variablen
- Klassenvariablen

Was lernen Sie kennen?

- Unterschiedliche Formen von Variablen
- Unterschied zwischen Sichtbarkeit und Lebensdauer von Variablen

Block (Wiederholung)

- Ist eine Programmeinheit, in der lokale Vereinbarungen getroffen werden können
- In Java beginnt ein Block mit **{** und endet mit **}**

```
public static void main(String[] args) {
    int x = 2, y = x;
    {
        int z = y + 2;
        x = z;
    }
    System.out.println(x + " " + y);
}
```

Block für z

Block für x und y

Hier sind nur x und y bekannt
Ausgabe: 4 2

Lokale Variablen (Wiederholung)

- Methoden können auch lokale Deklarationen enthalten
- Beispiel

```
public static void m(int x) {
        int y;
        short s;
        …
}
```

- Deklariert **3 lokale** Variablen y, s und x
  - Dürfen nur in dieser Methode verwendet werden
  - x wird wie eine lokale Variable behandelt

Klassenvariablen

- Variablen außerhalb von Methoden deklarieren
- Beispiel

```
public class Test1 {
    private static int a, b;
    private static void m(int x){…}
    public static void main(String[] args) {…}
}
```

- a und b sind Klassenvariablen (statische Variablen)
  - Durch `static` gekennzeichnet (meist `private`)
  - Können in **allen** Methoden der Klasse benutzt und verändert werden

Speicherplatz

Lokale Variablen

- Bei jedem neuen Aufruf der Methode wird Speicherplatz angelegt
- Am Ende der Methode wird der Speicherplatz wieder freigegeben

Klassenvariablen

- Speicherplatz wird beim Programmstart angelegt bzw. beim Programmende wieder freigegeben
- Werte existieren über Methodenaufrufe hinweg

Beispiel (Aufsummieren mit Klassenvariable)

```
public class CumulativeAdd {

    private static int sum = 0;

    private static void add(int x) {
        sum = sum + x;
    }

    public static void main(String[] arg) {
        add(1);
        add(2);
        add(3);
        System.out.println("sum = " + sum);
    }

}
```

Statische Variable

Aktueller Wert von sum wird verwendet

```
sum = 6
```

Sichtbarkeitsbereich (Sichtbarkeit)

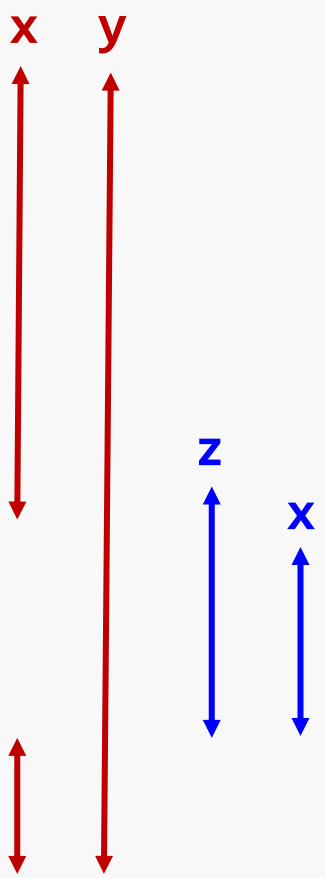
Jener Teil des Programms, in dem auf einen Bezeichner zugegriffen werden kann

Regeln (vollständige Liste siehe Spezifikation)

- Ein Bezeichner darf in einem Block nicht mehrmals deklariert werden (auch nicht in geschachtelten Anweisungsblöcken)
- Lokale Bezeichner verdecken Bezeichner, die auf Klassenebene deklariert wurden
- Der Sichtbarkeitsbereich eines lokalen Bezeichners beginnt bei seiner Deklaration und geht bis zum Ende der Methode bzw. bis zum Ende des umschließenden Blocks
- Auf Klassenebene deklarierte Bezeichner sind in allen Methoden der Klasse sichtbar

Sichtbarkeit – Beispiel

```
public class Example {
    private static void m1() {
        ...
    }
    private static int x;
    private static int y;
    private static void m2(int z) {
        int x;
        ...
    }
    ...
}
```



Sichtbarkeit – größeres Beispiel

```
public class Example {
    private static void method1() {
        System.out.println(x);
    }
    private static int x = 0;
    public static void main(String[] arg) {
        System.out.println(x);
        int x = 1;
        System.out.println(x);
        method1();
        if (x > 0) {
            int x;
            int y;
            ...
        } else {
            int y;
            ...
        }
        for (int i = 0; ...) {...}
        for (int i = 1; ...) {...}
    }
}
```

Gibt 0 aus

Gibt 0 aus

Gibt 1 aus

Geht nicht - x wurde in main schon deklariert

Kein Konflikt mit y im if-Zweig

Kein Konflikt mit i aus vorheriger Schleife

Lebensdauer

Lebensdauer ≠ Sichtbarkeit	Hängt davon ab, wo eine Variable deklariert wird
• Variable kann existieren, aber nicht sichtbar sein (z. B. x auf Klassenebene aus vorheriger Folie)	• Klasse – Vom Programmanfang bis zum Programmende • Lokal – Abhängig vom Block

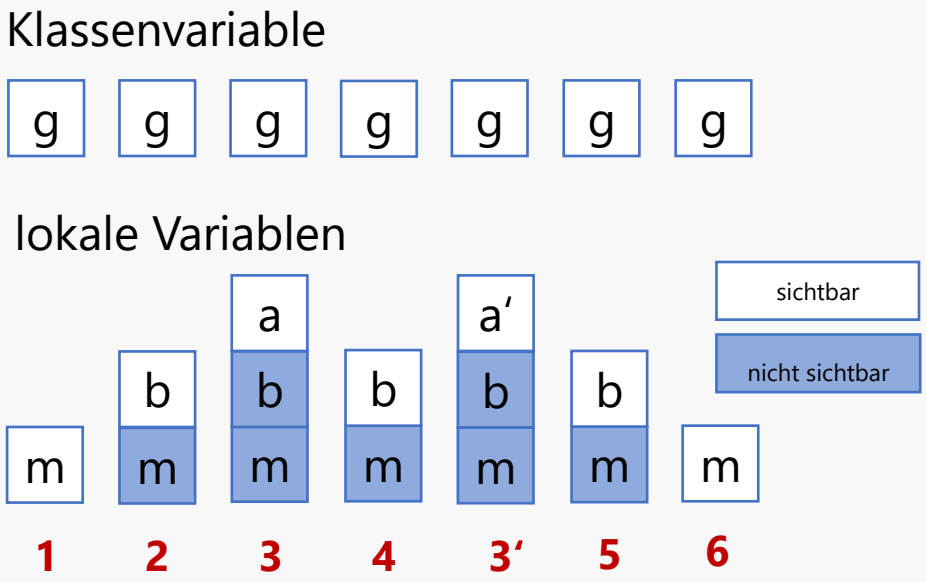
Lebensdauer – Beispiel

```
public class Tester {
    private static int g;

    public static void main(String[] args) {
        int m;
        1 … method2(); … 6
    }

    private static void method2() {
        int b;
        2 … method1(); … 4 … method1(); … 5
    }

    private static void method1() {
        int a;
        … 3 …
    }
}
```



Vorteile lokaler Variablen

Vereinfachen Wahl der Bezeichner

Fördern Lesbarkeit

- Deklaration und Verwendung der Variablen sind nah beieinander

Fördern Verständlichkeit

- Jede Methode ist für sich alleine überprüfbar

Fördern Sicherheit

- Lokale Variablen können nicht durch andere Methoden verändert werden
- Vermindern Seiteneffekte bei Klassenvariablen

Wenn möglich, Variablen **immer lokal** deklarieren